

„Soll das heißen, es besteht das Risiko, dass wir auf den Knopf drücken und den ganzen Hörsaal vernichten?“

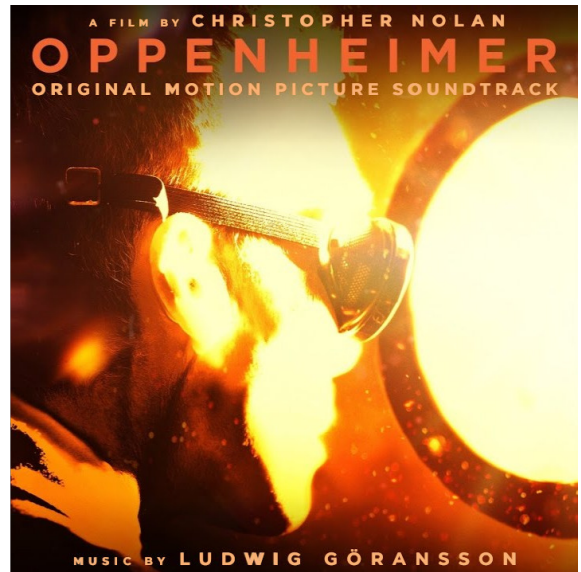
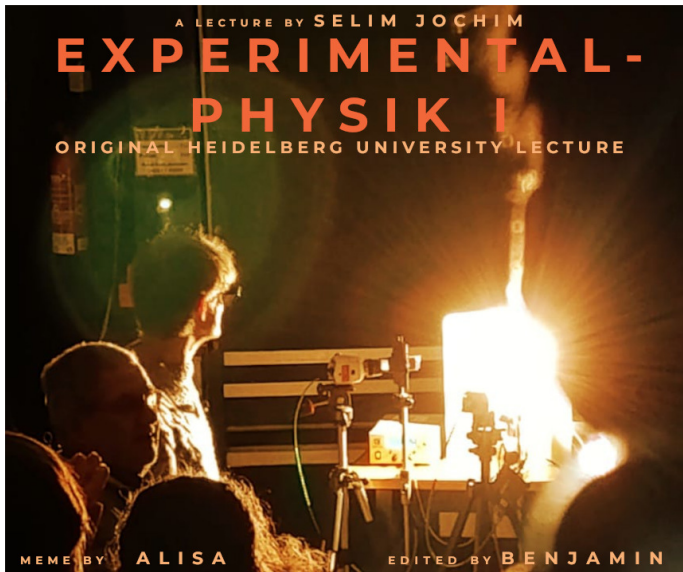


Abb. 0.1: Danke an Alisa für das Foto und die Oppenheimer⁷ Idee

Auswertung Versuch 252 - Aktivierung mit thermischen Neutronen

Physikalisches Praktikum PAP 2.2

des Studienganges Physik
an der Universität Heidelberg

von

Benjamin Kleijn

11. August 2026

Versuchstermin 16.06.2026
Tutor:in Lewin Dißelmeyer

Inhaltsverzeichnis

1. Einleitung	4
1.1. Motivation	4
1.2. Ziele des Versuches	4
1.3. Geräte	4
1.4. Theoretische und technische Grundlagen	6
1.4.1. Kritikalität	6
1.4.2. Erzeugung von Neutronenstrahlung	6
1.4.3. Abbremsen der Neutronen (thermische Neutronen)	7
1.4.4. Aktivierung von Silber (Erzeugung von Silberisotopen)	7
1.4.5. Aktivierung von Indium (Erzeugung von Indiumisomeren)	8
1.4.6. Zerfallskarte	9
2. Durchführung	10
2.1. Messverfahren	10
2.2. Messprotokoll	10
3. Auswertung	12
3.1. Zerfall von aktivierten Silberisotopen	14
3.2. Zerfall des aktivierten Indiumnuklids	15
4. Diskussion	17
4.1. Zerfallskonstanten der Silberisotope	17
4.2. Halbwertszeiten der Silberisotope	18
4.3. Halbwertszeit des Indiumisotops ^{116}In	18
4.4. Vergleich der Silber- und Indiummessung	19
4.5. Startaktivitäten	20
4.6. <i>Update:</i> Sättigungsaktivität	21
4.7. aufmodulierte Störung	22
4.8. Fazit	22
Literaturquellen	24
Abbildungsquellen	24
A. Code-Anhang	24

Abbildungsverzeichnis

0.1. <i>Meme</i> : Selim Jochim - Oppenheimer ⁷	1
1.1. Versuchsaufbau/Geräte ¹	5
1.2. Aktivierung von Silberisotopen ¹	8
1.3. Nuklidkartenausschnitt ¹	9
3.1. Fit der Zerfallsfunktion für Silberisotope	15
3.2. Fit der Zerfallsfunktion für Indiumisotope	16
4.1. <i>Meme</i> : Subtle Foreshadowing ⁹	17
4.2. Silber- und Indium Aktivität mit eingezeichnetem Untergrund	20
4.3. <i>Meme</i> : I'm tired, boss ¹⁰	23

Tabellenverzeichnis

4.1. ^{108}Ag und ^{110}Ag : Vergleich von $T_{1/2,\text{exp}}$ und $T_{1/2,\text{Lit}}$	18
4.2. ^{116}In : Vergleich von $T_{1/2,\text{exp}}$ und $T_{1/2,\text{Lit}}$	18

1. Einleitung

1.1. Motivation

Die Untersuchung radioaktiven Zerfalls hat grundlegende Bedeutung für das Verständnis von Kernprozessen und ermöglicht Anwendungen in verschiedenen Bereichen der Physik, Medizin und Technik, deswegen wird in diesem Versuch der Zerfall von unterschiedlichen Isotopen untersucht.

Genutzt werden in diesem Versuch durch Kernspaltung aktivierte Isotope: Auf diesem Prinzip der Kernspaltung beruhen Kernkraftwerke und Atombomben: Ein schwerer Atomkern spaltet sich unter Energiefreisetzung und typischerweise Emission von schnellen Neutronen in zwei oder mehr kleinere Spaltprodukte. Diese Spaltung wird durch thermische (langsame) Neutronen induziert, die aus schnellen Neutronen durch Abbremsen mithilfe des *Moderators* gewonnen werden können, und somit ist unter bestimmten Voraussetzungen eine Kettenreaktion möglich, bei der das radioaktive Material in einen *verzögert kritischen* bzw. beim Hochfahren des Reaktors *verzögert überkritischen* (Kernkraftwerke) oder *prompt überkritischen* Zustand (Atombomben) überführt werden kann. Dies dient der Energiegewinnung und Produktion von radioaktiven Materialien bei kommerziellen Kernreaktoren, Forschungszwecken bei Forschungsreaktoren und Massenvernichtung bei Atombomben.

Es macht also Sinn, die grundlegenden Eigenschaften von radioaktiven Isotopen wie Zerfall und Aktivierung zu untersuchen.

1.2. Ziele des Versuches

Durch Neutronenbeschuss kann ein stabiles Isotop zu einer radioaktiven Quelle aktiviert werden, welches dann mit einer gewissen Halbwertszeit zerfällt. In diesem Versuch stehen zur Untersuchung das aktivierte Indium(In)-Isotop ^{116}In und die aktivierten Silber(Ag)-Isotope ^{108}Ag und ^{110}Ag zur Verfügung, um folgendes zu erreichen:

- Bestimmung der Halbwertszeit von ^{116}In
- Bestimmung der Halbwertszeit von ^{108}Ag und ^{110}Ag
- durch Anfitzen der Zerfallsfunktion an die aufgenommenen Messdaten

1.3. Geräte

Eine Neutronenquelle, bestehend aus einem Gemisch von Berylliumspänen und dem α -Strahler Americium-241, wird von einem moderierenden Paraffinblock umgeben, der die schnellen Neutronen auf thermische Energien abbremsst. Die zu aktivierenden Proben (Silber- und Indiumbleche) werden in einer Probenhalterung positioniert und während der Aktivierungsphase in die

Nähe der Neutronenquelle gebracht. Nach der Aktivierung wird das Präparat vor ein Geiger-Müller-Zählrohr gespannt, welches über ein Betriebsgerät bedient und ausgelesen werden kann. Der Impulszähler zählt die registrierten Zerfälle über jeweils festgelegte Zeitintervalle (Torzeiten). Die Daten werden auf einem PC gesammelt und anschließend mit geeigneter Software ausgewertet.

1. Geiger-Müller-Zählrohr mit Betriebsgerät
2. Externer Impulszähler
3. Neutronenquelle
4. Präparatehalterung
5. Indium- und Silberbleche

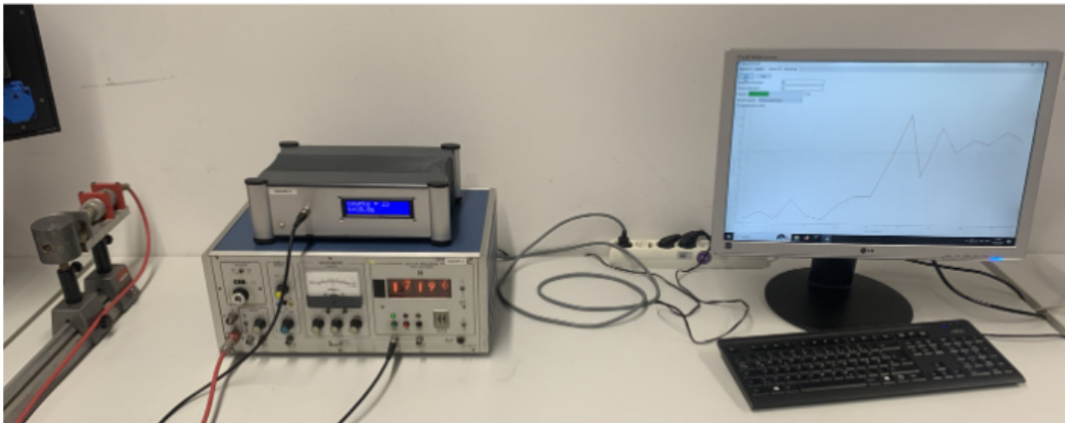


Abb. 1.1: Versuchsaufbau/Geräte¹

1.4. Theoretische und technische Grundlagen¹

1.4.1. Kritikalität^{2,8}

Für eine Kettenreaktion gibt es zwei Bedingungen: Es muss genügend Material/Atomkerne vorhanden sein, sodass auch jedes freigesetzte Neutron wieder zur einer weiteren Spaltung führt, dies wird durch die *kritische Masse* beschrieben, die von der Geometrie und der Anzahl an Atomkernen abhängt. Und zudem muss die Neutronenbilanz, also wie viele spaltungsfähige Neutronen pro spaltungsfähigem Neutron erzeugt werden, entsprechend ausgestaltet sein. Es macht mehr Sinn von einer Spaltungsbilanz zu sprechen, da pro Neutron/Spaltung mehrere Neutronen erzeugt werden, die jedoch nicht alle wieder zu einer weiteren Spaltung führen (keine kritische Masse, technische Effekte), deswegen dient der Multiplikationsfaktor k eher dazu, die Anzahl neuer Spaltungen pro gespaltenem Kern zu beschreiben. Mehr als 99 % aller Neutronen, die *prompten* Neutronen, werden auf einer sehr geringen Zeitskala von $t < 10^{-14}$ s emittiert, die restlichen sind sogenannten *verzögerten* Neutronen die zu k den Faktor β beitragen und erst nach ein paar Millisekunden bis Minuten emittiert werden. Sie sind somit auf einer Zeitskala, auf welche mithilfe von technischen Mitteln reagiert werden kann, in der Steuerung von k und β liegt die Kunst der Reaktorregelung:

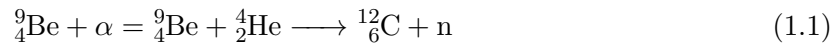
- $k < 1$ unterkritisch, Neutronenverlust überwiegt
- $k = 1$ verzögert kritisch, ausgeglichene Spaltungsbilanz.
- $1 < k < 1 + \beta$ verzögert überkritisch, Neutronenerzeugung, aber nur durch verzögerte Neutronen, überwiegt -verlust. Regelbarer Bereich
- $k = 1 + \beta$ prompt kritisch, jede minimale Schwankung lässt das System direkt in die nächste Stufe übergehen
- $k > 1 + \beta$ prompt überkritisch, exponentielle Kettenreaktion, Reaktorexlosion bzw. nukleare Explosion einer Atombombe

Für den hiesigen Versuch ist es leider nicht sehr relevant, aber die technischen Mittel/Rückkopplungen um Überkritikalität erst zu vermeiden/rückgängig zu machen, sind sehr interessant.

1.4.2. Erzeugung von Neutronenstrahlung

Eine radioaktive Quelle kann durch Aktivierung stabiler Isotope mithilfe von Kernreaktionen hergestellt werden. Für die Anregung solcher Kernreaktionen werden häufig Neutronen verwendet, da diese elektrisch neutral sind und somit nicht der Coulomb-Wechselwirkung unterliegen. Der entsprechende Atomkern kann ein Neutron daher besonders leicht einfangen.

Für diesen Versuch wird eine Neutronenquelle eingesetzt, die aus einem Gemisch von Berylliumspänen und dem α -Strahler Americium-241 (^{241}Am) besteht. Es tritt folgende Kernreaktion auf:



Bei dem Beschuss von Beryllium mit α -Teilchen entsteht Kohlenstoff sowie ein freies Neutron. Das so erzeugte Neutron besitzt eine hohe kinetische Energie im Bereich von etwa (1 bis 10) MeV.

1.4.3. Abbremsen der Neutronen (thermische Neutronen)

Die bei der oben beschriebenen Reaktion entstehenden schnellen Neutronen müssen zunächst abgebremst werden, damit sie für bestimmte Kernreaktionen nutzbar gemacht werden können. Je langsamer ein Neutron ist, desto höher ist die Wahrscheinlichkeit, dass es von einem Atomkern eingefangen wird. Der Abbremsvorgang erfolgt hier durch einen Paraffinblock, der die Neutronenquelle umgibt.

Paraffin als Moderator ist für diesen Prozess besonders geeignet, da es zahlreiche Wasserstoffkerne (Protonen) enthält. Trifft ein Neutron auf ein Proton, kommt es zu einem elastischen Stoß, bei dem das Neutron etwa die Hälfte seiner kinetischen Energie verliert. Bei Stößen mit Kohlenstoffkernen im Paraffin geht dagegen weniger Energie verloren, da die Kohlenstoffkerne deutlich größer als die Neutronen sind. Die nach diesem Prozess abgebremsten Neutronen besitzen eine nahezu thermische Energie und werden als thermische Neutronen bezeichnet.

1.4.4. Aktivierung von Silber (Erzeugung von Silberisotopen)

Natürliches Silber³ setzt sich zusammen aus 51.893 % ^{107}Ag und 48.161 % ^{109}Ag . Wird natürliches Silber wie oben erläutert aktiviert, entsteht aus ^{107}Ag das Isotop ^{108}Ag und aus ^{109}Ag das Isotop ^{110}Ag . Beide entstandenen Isotope stellen β^- -Strahler dar, die durch Zerfall in ^{108}Cd bzw. ^{110}Cd übergehen.

^{110}Ag hat eine um den Faktor sechs kürzere Halbwertszeit und erreicht daher den Sättigungswert, wenn sich Erzeugung von N radioaktiven Atomkernen und Zerfall $Z \sim N$ ausgleichen, vor ^{108}Ag . Bei längerer Aktivierungszeit t' dominiert ^{108}Ag aufgrund seiner deutlich längeren Halbwertszeit, da von ^{110}Ag bei längeren Aktivierungszeiten die maximale Anzahl durch den Sättigungswert schon erreicht ist. Bei kurzen Aktivierungszeiten wird hingegen hauptsächlich ^{110}Ag gebildet. Das Aktivitätsverhältnis der beiden Isotope verändert sich also mit der Bestrahlungszeit während der Aktivierung. In Abb. 1.2 ist als gestrichelte Linie die im Versuch genutzte Aktivierungszeit von 7 min eingezeichnet. Hier stehen die zwei Isotope annähernd im selben Verhältnis.

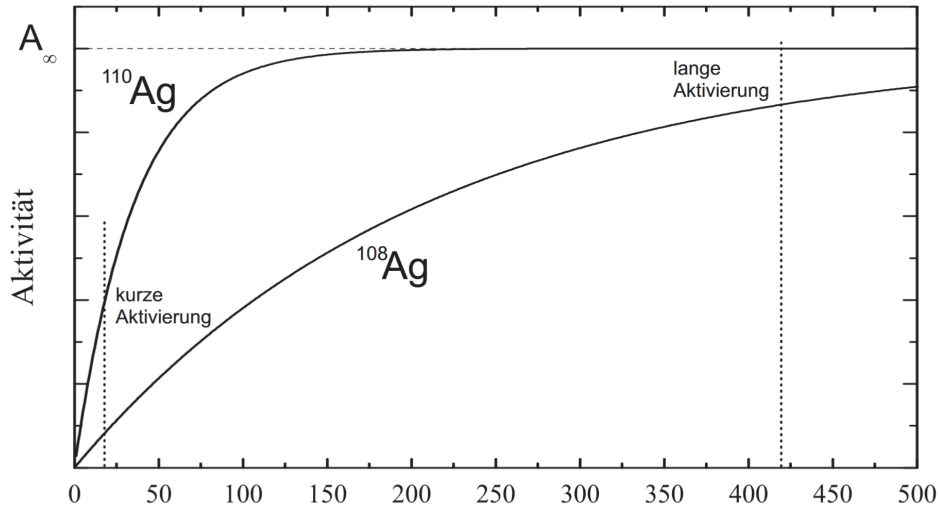


Abb. 1.2: Aktivierung der natürlich vorkommenden Silberisotope 51 % ^{107}Ag und 49 % ^{109}Ag ¹

Während des Aktivierungsprozesses werden kontinuierlich neue radioaktive Kerne gebildet, während gleichzeitig bereits vorhandene zerfallen. Die Aktivität $A(t')$ (Zahl der Zerfälle pro Sekunde) folgt dem Gesetz:

$$A(t') = A_{\infty} (1 - e^{-\lambda t'}) \quad (1.2)$$

Dabei bezeichnet A_{∞} die maximale Aktivität im Gleichgewicht (Sättigungsaktivität), λ die Zerfallskonstante des Isotops und t' die Aktivierungszeit. Für $t' \rightarrow \infty$ nähert sich $A(t')$ asymptotisch an A_{∞} an.

1.4.5. Aktivierung von Indium (Erzeugung von Indiumisomeren)

Durch Aktivierung von stabilem Indium kommt es zur Bildung zweier Isomere eines radioaktiven Indiumisotops. Zum einen entsteht das Isotop ^{116}In , welches sich im energetischen Grundzustand befindet, zum anderen ^{116m}In , welches sich im metastabilen Zustand befindet. Beide Isotope sind β^- -Strahler, zeigen aber unterschiedliche Halbwertszeiten. Beide Isotope des Indiums zerfallen zu ^{116}Sn (Zinn).

Nach Abschluss der Aktivierung beginnt der rein radioaktive Zerfall, da keine neuen radioaktiven Kerne mehr erzeugt werden. Dann folgt:

$$A(t) = A_0 \cdot e^{-\lambda t} \quad (1.3)$$

Für die Halbwertszeit gilt abhängig der spezifischen Zerfallskonstante λ :

$$T_{1/2} = \frac{\ln(2)}{\lambda} \quad (1.4)$$

Zusätzlich kann auch die mittlere Lebensdauer τ definiert werden als:

$$\tau = \frac{1}{\lambda} \quad (1.5)$$

Durch Messung der Zerfallsrate über die Zeit lässt sich die Zerfallskonstante bestimmen, woraus sich gemäß Gleichung (1.4) die Halbwertszeit berechnen lässt. Dies bildet die Grundlage der experimentellen Auswertung.

1.4.6. Zerfallskarte

In folgendem Bild sieht man den Ausschnitt der Nuklidkarte mit entsprechenden Zerfallsarten, Elemente und Massenzahlen, sowie Energien und Halbwertszeiten.

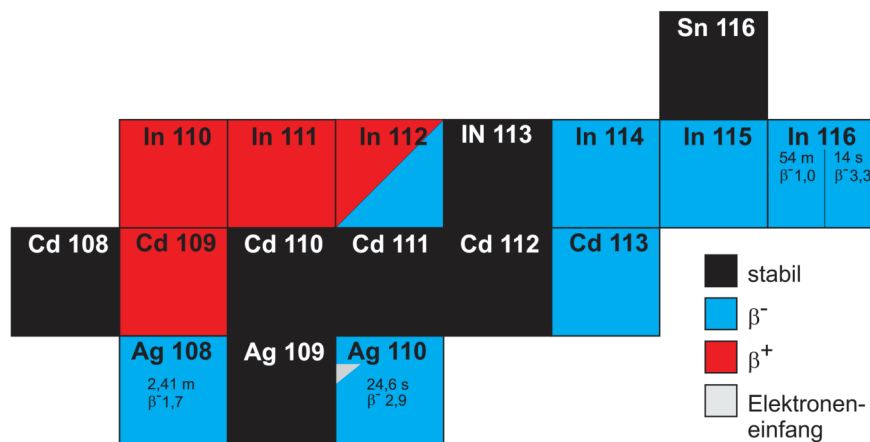


Abb. 1.3: Ausschnitt der Nuklidkarte mit den Halbwertszeiten und der Energie der emittierten Strahlung¹

2. Durchführung

2.1. Messverfahren

Messung von Silber Für die Messung der Halbwertszeit von aktiviertem Silber wird eine Geiger-Müller-Zählrohrspannung zwischen 500 V und 550 V am Betriebsgerät eingestellt. Zunächst wird die Untergrundzählrate bestimmt, um diese später von den Messwerten abziehen zu können. Dazu wird die Torzeit t auf $t = 10$ s eingestellt und 50 Messungen durchgeführt, sodass die Untergrundzählrate über einen Zeitraum von etwa acht Minuten erfasst wird. Die Torzeit ist die Dauer einer Einzelmessung, in der alle in dieser Zeit aufgenommenen Counts N addiert werden und somit letztlich die Messgröße $\frac{N}{t}$ ausgegeben wird.

Anschließend wird die eigentliche Silbermessung durchgeführt. Die Träger mit den Silberblechen werden zur Aktivierung für sieben Minuten in der Neutronenquelle platziert. Nach Beendigung der Aktivierung wird das Präparat schnellstmöglich an das Zählrohr gebracht und dort mit einem Aluminiumblech fixiert. Die Messung startet mit einer Torzeit von 10 s über eine Gesamtzeit von 400 s. Dieser Zyklus wird insgesamt viermal wiederholt, um die statistische Unsicherheit zu reduzieren. Die vier Messreihen werden anschließend addiert und gemeinsam ausgewertet.

Messung von Indium Für die Messung der Halbwertszeit von aktiviertem Indium wird analog vorgegangen. Das Messintervall wird dabei auf 120 s eingestellt. Das aktivierte Indium-Präparat wird in der Halterung fixiert und über einen Zeitraum von etwa 50 Minuten gemessen. Auch hier wird zuvor die Untergrundzählrate bestimmt, wobei die Torzeit entsprechend angepasst wird. Da bei der Aktivierung von Indium neben dem Grundzustand auch ein metastabiler Zustand mit sehr kurzer Halbwertszeit gebildet wird, wird der erste Messpunkt für den Fit vernachlässigt, da dieser vom eigentlichen Zerfall der betrachteten Isotope abweicht.

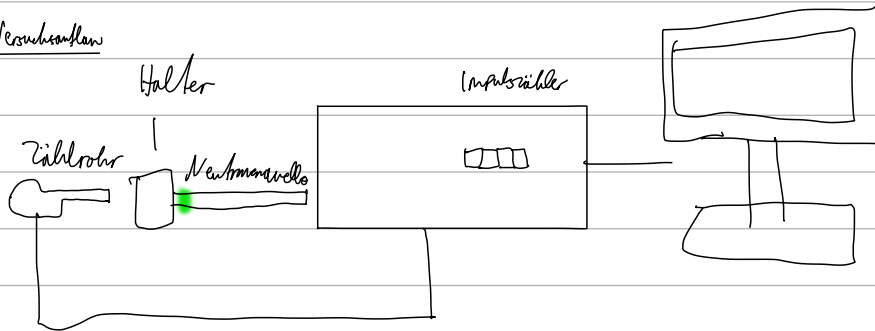
2.2. Messprotokoll

Messgeräte

- Geiger-Müller Zählrohr mit Betriebsgerät
- externer Impulszähler
- PC
- Präparathaltung
- Neutronenquelle

PC

Versuchsskizze



Aufgabe 1: Messen der Halbwertszeit von aktiviertem Indium

Wir stellen am Zählrohr eine Spannung von (500 ± 20) V ein. Zunächst messen wir die Untergrundzählrate bei einer Totzeit von $t = 10$ s über einen Zeitraum von etwa 8 min (48 Messungen). Die Daten speichern wir ab.

Danach stecken wir das aktive Indium-Präparat in die Halterung, und stellen die Totzeit auf $t = 120$ s. Für 50 min messen wir die Counts.

Aufgabe 2: Messen der Halbwertszeit von aktiviertem Silber

Wir verwenden eine Totzeit von $t = 10$ s. Wir aktivieren diese 7 min lang, und fixieren es vor dem GM-Zähler. Wir messen für 900 s die Counts, und führen die Messung für 4 Aktivierungszyklen durch.

16.06.20
W

3. Auswertung

Formeln der statistischen Analyse¹

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{S.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{S.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{S.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{S.4})$$

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \dots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{S.5})$$

$$\text{relativer Fehler von } f = \prod_{i=1}^N x_i^{\alpha_i} \quad \frac{\Delta f}{|f|} = \sqrt{\sum_{i=1}^N \left(\frac{\alpha_i \Delta x_i}{x_i} \right)^2} \quad (\text{S.6})$$

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

$$\text{Absolute Abweichung} \quad A = |a_{\text{lit}} - a_{\text{exp}}|$$

$$\Delta A = \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2} \quad (\text{S.7})$$

$$\text{Relative Abweichung} \quad R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100 \quad (\text{S.8})$$

$$\text{Signifikanz} \quad S[\sigma] = \frac{A}{\Delta A} = \frac{|a_{\text{lit}} - a_{\text{exp}}|}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}} \quad (\text{S.9})$$

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1, \dots, a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_{i=1}^n \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2 \quad (\text{S.10})$$

$$\text{Freiheitsgrade} \quad f = n - m \quad (\text{S.11})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{S.12})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{S.13})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. Zerfall von aktivierten Silberisotopen

Die Aktivität A wird in der Einheit Becquerel $[\text{Bq}] = [\text{s}^{-1}]$ gemessen, um somit konsistent zu bleiben, müssen alle erhobenen Messungen der Zerfälle N pro Torzeit T in Zählraten n umgerechnet werden:

$$n = \frac{N}{T} \qquad \Delta n = \frac{\sqrt{N}}{T} \qquad (3.1)$$

Die 4 Messreihen N_i einer Gesamtdauer von $t = 400\text{ s}$ bei Torzeit $T = 10\text{ s}$ (jede dieser Messreihen besteht also aus 40 Werten) werden zu einer Gesamtanzahl an Zerfällen N addiert und durch die vierfache Torzeit geteilt, um bei der realen Zählrate zu bleiben und keine vierfache zu betrachten. Der Fehler wird gemäß der dem radioaktiven Zerfall zugrundeliegenden Poissonverteilung bestimmt:

$$n = \frac{1}{4T} \sum_{i=1}^4 N_i = \frac{N}{4T} \qquad \Delta n = \frac{\sqrt{\sum_{i=1}^4 N_i}}{4T} \qquad (3.2)$$

Die Untergrundmessung bei Torzeit $T = 10\text{ s}$ und Messdauer von 8 min (besteht also aus 48 Werten N_{0_i}) wird jetzt nicht mit dem Faktor 4 multipliziert, da die Zählraten oben schon durch $4T$ wieder heruntergerechnet wurden. n_0 wird durch arithmetische Mittelung bestimmt:

$$n_0 = \frac{1}{48 \cdot T} \sum_{i=1}^{48} N_{0_i} \qquad \Delta n_0 = \frac{\sigma(N_{0_i})}{T \cdot \sqrt{48 - 1}} \qquad (3.3)$$

Man beachte: n_0 ist ein Skalar, n ein Array. Der Fehler Δn_0 bestimmt sich durch den Fehler des Mittelwerts (Gleichung (S.1)).

Diese Messreihe $n(t)$ kann nun geplottet und eine entsprechende Funktion für die Zahl der Zerfälle Z angefitet werden, die aus der Superposition der einzelnen Zerfallsmodelle für ^{108}Ag und ^{110}Ag sowie dem Untergrund ausgeht:

$$Z(t) = A_{108} \cdot e^{-\lambda_{108}t} + A_{110} \cdot e^{-\lambda_{110}t} + n_0 \qquad (3.4)$$

Es werden nun zur Fehlerbestimmung von λ sukzessive Fits durchgeführt, die sich in der Wahl von n_0 unterscheiden:

$$\lambda = \lambda_{n_0} \qquad \Delta\lambda = \sqrt{\text{Mean}(|\lambda_{n_0} - \lambda_{n_0 \pm \Delta n_0}|)^2 + \Delta\lambda_{n_0}^2} \qquad (3.5)$$

Hierbei sind die indizierten λ die Fitparameter-Ergebnisse bei fester n_0 Wahl und $\Delta\lambda_{n_0}$ der Fitfehler aus der Kovarianzmatrix für den Fit mit reinem Untergrundwert n_0 . Man erhält damit aus den Fits in Abb. 3.1 die Ergebnisse für λ und nach Umrechnung in Halbwertszeiten

$T_{1/2}$ und Lebensdauern τ mittels:

$$T_{1/2} = \frac{\ln 2}{\lambda} \quad \Delta T_{1/2} = \ln(2) \frac{\Delta \lambda}{\lambda^2} \quad (3.6)$$

$$\tau = \frac{1}{\lambda} \quad \Delta \tau = \frac{\Delta \lambda}{\lambda^2} \quad (3.7)$$

$\lambda_{110} = (0.030 \pm 0.008) \text{ s}^{-1}$	$T_{110} = (23 \pm 7) \text{ s}$	$\tau_{110} = (33 \pm 9) \text{ s}$
$\lambda_{108} = (0.0048 \pm 0.0024) \text{ s}^{-1}$	$T_{108} = (140 \pm 80) \text{ s}$	$\tau_{108} = (420 \pm 110) \text{ s}$

Aus der Einleitung bzw. dem Skript ist bekannt, dass ^{108}Ag eine sechsfach höhere Halbwertszeit $T_{1/2}$ hat. Dementsprechend konnten die Halbwertszeiten und Zerfallskonstanten identifiziert werden.

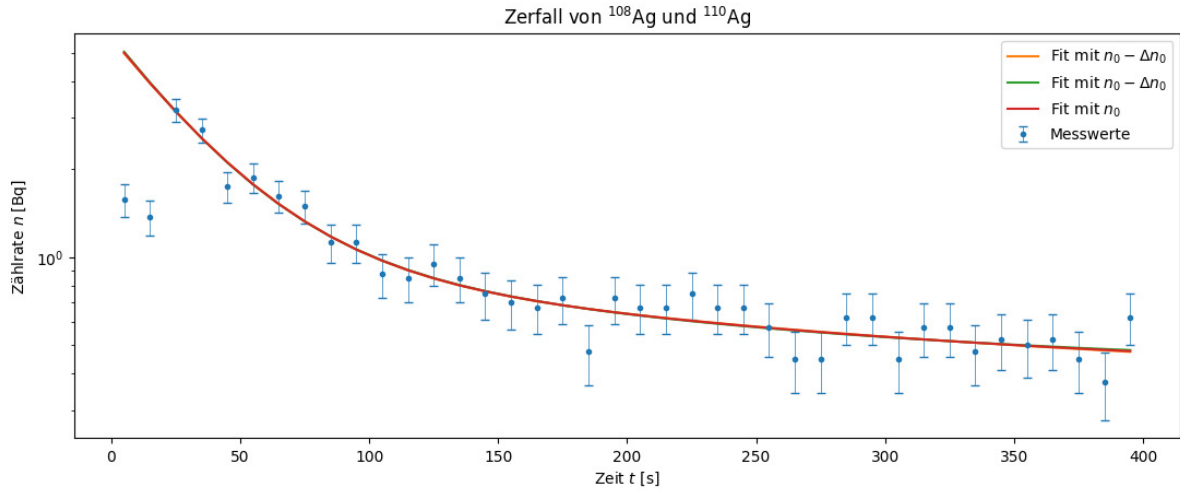


Abb. 3.1: Fit der Zerfallsfunktion für Silberisotope

Fitgüte: Die Güte des Fits lässt sich gemäß den Größen aus Abschnitt 3.(Formeln der statistischen Analyse¹) beurteilen:

$$\chi^2 = 19.0 \quad \chi^2_{\text{red}} = 0.6 \quad p = 98 \%$$

3.2. Zerfall des aktivierten Indiumnuklids

Bei dieser Messung wird analog vorgegangen, jedoch existiert diesmal nur eine Messreihe und die Untergrundmessung. Die Messreihe wird analog wie oben geplottet und analysiert, mit einzigem Unterschied in der angefitzten Funktion:

$$Z(t) = A_{116} \cdot e^{-\lambda_{116}t} \quad (3.8)$$

Man erhält damit aus den Fits in Abb. 3.2 die Ergebnisse für λ_{116} , $T_{1/2}$ und τ :

$$\lambda_{116} = (0.000\,218 \pm 0.000\,012) \text{ s}^{-1} \quad T_{116} = (3180 \pm 180) \text{ s} \quad \tau = (4600 \pm 300) \text{ s}$$

Fitgüte: Es lässt sich wiederum analog die Güte des Fits beurteilen:

$$\chi^2 = 30.0 \quad \chi^2_{\text{red}} = 1.7 \quad p = 3.0 \%$$

Das sind wie auch oben schon schlechte Werte, optimalerweise sollte $p = 50 \%$ betragen. Die Gründe werden später diskutiert.

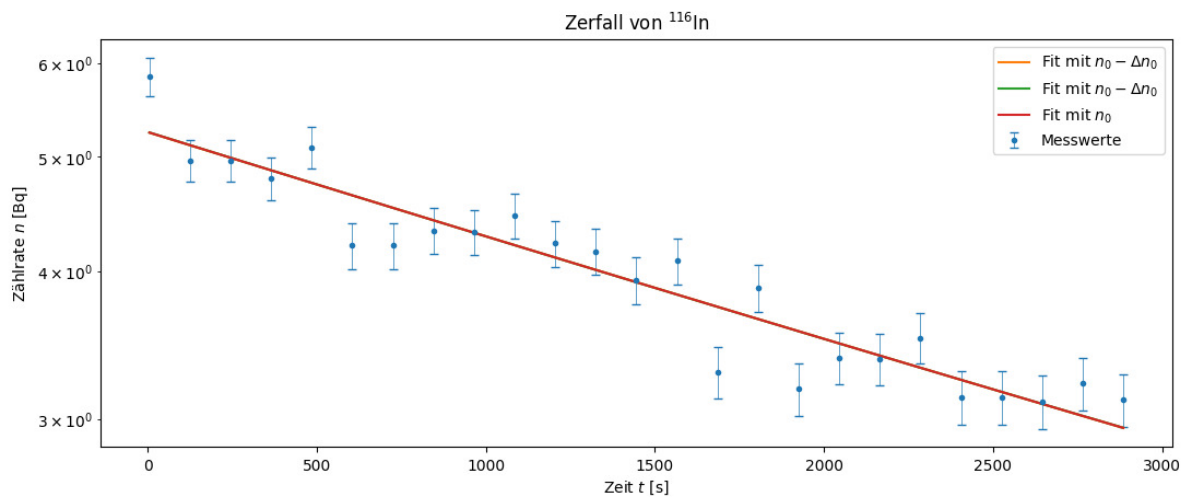


Abb. 3.2: Fit der Zerfallsfunktion für Indiumisotope

4. Diskussion

4.1. Zerfallskonstanten der Silberisotope

Die ersten zwei Messpunkte in Abb. 3.1 wurden nicht zur Fitbildung einbezogen. Eine Ursache für diese Ausreißer kann ich mir nicht vorstellen, ein rein statistischer Zufall wirkt unwahrscheinlich aufgrund des doppelten Auftretens.

Außerdem wurden die Werte A_i nicht extra in diesem Dokument angezeigt, da sie für die Auswertung irrelevant sind.



Abb. 4.1: Zu früh gefreut⁹

Fitfehler: Bevor man die Anpassung der Fehler mithilfe der Δn_0 Fits vornimmt, kommt man auf $\tilde{\lambda}_{110} = (0.030 \pm 0.008) \text{ s}^{-1}$ und $\tilde{\lambda}_{108} = (0.0048 \pm 0.0023) \text{ s}^{-1}$. Im Vergleich mit den Ergebnissen für die endgültigen λ ist damit der hauptsächliche Teil des Fehlers vom ursprünglichen Fitfehler bestimmt, der bei ^{108}Ag bei fast $\frac{\Delta\lambda_{n0}}{\lambda_{n0}} = 50\%$ liegt. Diese Zahlen werden auch durch die in Abb. 3.1 zu sehenden Fitkurven bei verschiedenen Untergrundwerten unterstützt: Es ist ohne Zoomen im Ausgabefenster kein Unterschied zwischen dem Verlauf der Kurven erkennbar, weder bei der Silber- noch bei der Indiumauswertung. Die Dominanz des ursprünglichen Fehlers $\Delta\lambda_{n0}$ liegt eventuell daran, dass wie in Abb. 4.2 zu sehen, sehr viele Messwerte vorliegen, die einfach nur noch den Untergrund messen, und Anfangswerte mit hohen Zerfallszahlen fehlen, mithilfe derer die Kurvencharakteristik und damit die Zerfallskonstante besser zu bestimmen wäre. Der klare Schluss hieraus ist, sehr viel mehr Wert auf zügiges Arbeiten zu legen: Hauptsächlich durch zügige Entnahme, Transport und Einspannung der aktivierten Probe kann Zeit gespart werden, ein kleiner Teil kann auch durch präventives Starten der Messung beigetragen werden, indem diese gestartet wird, bevor das Präparat eingelegt wird. Die nicht relevanten Werte können dann in der Auswertung problemlos aus dem Array entfernt werden.

Ich kann mich leider nicht mehr daran erinnern, ob hier irgendwie außerordentlich langsam gearbeitet wurde und damit explizit menschliche Versagen Fehlerursache ist (ist es eigentlich sowieso immer).

Fitgüte: Die Ungenauigkeit der Silber-Fitergebnisse liegt zu einem Teil auch an der hohen Zahl von Fitparametern, die Kopplung zweier Exponentialfunktion ist wsl. äußerst anfällig für kleine Schwankungen.

Es wirkt zwar durch reine visuelle Betrachtung des Fits nicht, als ob die Abweichung vom Messwertetrend stark ist, die χ^2_{red} Summe liegt jedoch bei 0.6, also weit weg vom optimalen Wert 1. Zwar ist der Fehlerbereich für λ sehr hoch, aber dies wirkt sich nicht auf die χ^2 -Summe aus, denn diese berücksichtigt nur die Abweichung zwischen Fitfunktion und echten Messwerten. Damit ist die obige Vermutung umso wahrscheinlicher, dass aufgrund fehlender charakteristischer Anfangswerte effektiv nur wenig Werte zum Berechnen des Fits zur Verfügung stehen und der Fit dadurch wohl weder in seinen Fitparametern noch den Fehlergrenzen optimal ist.

4.2. Halbwertszeiten der Silberisotope

Tabelle 4.1: ^{108}Ag und ^{110}Ag : Vergleich von $T_{1/2,\text{exp}}$ und $T_{1/2,\text{Lit}}$

Messreihe	$T_{1/2,\text{exp}}[\text{s}]$	$T_{1/2,\text{Lit}}[\text{s}]$	abs.Abw.[s]	proz.Abw.[%]	$S[\sigma]$
^{108}Ag	140 ± 80	144.6	0 ± 80	0 ± 60	0.06
^{110}Ag	23 ± 7	24.6	2 ± 7	10 ± 40	0.23

Die hohen Fehler der Halbwertszeit sorgen dafür, dass die Sigma-Abweichung nicht mehr sehr aussagekräftig ist, deswegen wäre als weitere Metrik die absolute Abweichung interessant, die nicht angezeigt wird, da sie aufgrund des hohen Fehlers auf 0 abgerundet wurde. Berechnet man die unsignifikanterte abs. Abweichung, so hält sich diese bei beiden Isotopen in Grenzen und ist nicht außergewöhnlich hoch.

4.3. Halbwertszeit des Indiumisotops ^{116}In

Man sieht in der logarithmisch aufgetragenen Abb. 3.2 die exponentielle Natur des einzelnen Zerfalls: Da keine Überlagerung von zwei verschiedenen Zerfallsreihen vorliegt, sondern nur der Zerfall von ^{116}In , ist der Verlauf von $Z(t)$ im logarithmischen Bild annähernd linear.

Tabelle 4.2: ^{116}In : Vergleich von $T_{1/2,\text{exp}}$ und $T_{1/2,\text{Lit}}$

$T_{1/2,\text{exp}}[\text{s}]$	$T_{1/2,\text{Lit}}[\text{s}]$	abs.Abw.[s]	proz.Abw.[%]	$S[\sigma]$
3180 ± 180	3248.4	70 ± 180	2 ± 6	0.4

Die Abweichung ist nicht statistisch signifikant, die Messung ist angesichts des deutlich geringeren relativen Fehlers $\frac{\Delta T_{1/2}}{T_{1/2}}$ der Halbwertszeiten um einiges genauer als die Silbermessung, die Gründe werden in folgendem Abschnitt erörtert.

4.4. Vergleich der Silber- und Indiummessung

Torzeitabhängigkeit des relativen Fehlers: In Versuch 251 - „Statistik des radioaktiven Zerfalls“ wurde explizit berechnet, wie durch Erhöhung der Messdauer der relative Fehler $\frac{\Delta n}{n}$ sinkt:

$$\frac{\Delta n}{n} = \frac{\sqrt{N}/T}{N/T} = \frac{1}{\sqrt{N}} = \frac{1}{\sqrt{n \cdot T}} = \text{constant} \cdot \frac{1}{\sqrt{T}} = \text{Mean} \left(\frac{1}{\sqrt{n_i}} \right) \cdot \frac{1}{\sqrt{T}} \quad (4.1)$$

Er skaliert also mit $\sim \frac{1}{\sqrt{T}}$ und die Torzeit liegt bei der Indiummessung mit $T = 120 \text{ s} \gg 10 \text{ s}$ weitaus höher als bei der Silbermessung. Insgesamt beträgt der relative Fehler für Silber 18 % und für Indium 5 %, ist also für Silber deutlich höher, da hier wegen kleiner Torzeit deutlich kleinere Zerfallszahlen gemessen wurden. Jedoch ist diese Messung leider alternativlos, denn man muss die Torzeit natürlich abhängig der Halbwertszeit des Isotops wählen: Die ist bei ^{110}Ag mit $T_{1/2} = 24.6 \text{ s}$ so gering, dass sich bei längerer Torzeit die Zählrate pro Torzeitintervall merkbar ändern würde, denn $n = n_0 \exp \left(-\frac{\ln(2)}{T_{1/2}} t \right)$.

Reduktion des rel. Fehlers von Silber: Es gibt jedoch einen Weg, wie der relative Fehler von Silber verringert werden kann: Indem m Messreihen aufgenommen und addiert werden, denn:

$$n = \frac{1}{m \cdot T} \sum_i^m N_i \quad \begin{matrix} N_i \approx N_j \\ \text{im Mittel} \end{matrix} \quad \frac{m N_i}{m T} = \frac{N_i}{T} \quad (4.2)$$

$$\Delta n = \frac{\sqrt{\sum_i^m N_i}}{m T} \quad \begin{matrix} N_i \approx N_j \\ \text{im Mittel} \end{matrix} \quad \frac{\sqrt{m N_i}}{m T} = \frac{1}{\sqrt{m}} \frac{\sqrt{N_i}}{T} \quad (4.3)$$

$$\Rightarrow \frac{\Delta n}{n} = \frac{1}{\sqrt{m}} \frac{1}{\sqrt{N_i}} = \frac{1}{\sqrt{m}} \frac{1}{\sqrt{n}} \frac{1}{\sqrt{T}} \quad (4.4)$$

D.h. wenn anstatt 4 Messreihen 16 gemacht werden, das Verhältnis der relativen Fehler von Silber und Indium beträgt ungefähr $\frac{1}{4} = \frac{1}{\sqrt{16}}$, dann sind die relativen Fehler von Silber und Indium auf demselben Niveau.

Untergrundvergleich: Der relative Fehler der Messpunkte, die für den Fit genutzt wurden, ist also wie diskutiert für Silber deutlich höher und hebt so die Fehlergrenzen der Ergebnisse der Fitparameter an. Zudem spielt natürlich die schiere Fitparameteranzahl (4 vs. 2) wieder eine Rolle sowie vor allem die zur Verfügung stehenden Messwerte: Wie oben beschrieben, wurde bei Silber viel Untergrund gemessen, was aus der berechneten Halbwertszeit direkt hervorgeht: $T_{1/2}^{110,108} = (23.7, 140) \text{ s} \ll t = 400 \text{ s}$ ist also deutlich kleiner als die insgesamt Messdauer, die Aktivität sinkt sehr schnell. Bei ^{116}In hingegen ist $T_{1/2}^{\text{In}} = (3450 \pm 190) \text{ s} > t = 3000 \text{ s}$ sogar

größer als die gesamte Messdauer. Damit ist die Anzahl der *effektiv* zur Verfügung stehenden Messwerte - die nicht schon auf Untergrundniveau liegen - zur Fitbildung bei Indium deutlich höher.

Zur besseren Erkenntnis wurden hier, bei nicht-logarithmischer Skalenteilung, die Untergrundniveaus eingezeichnet.

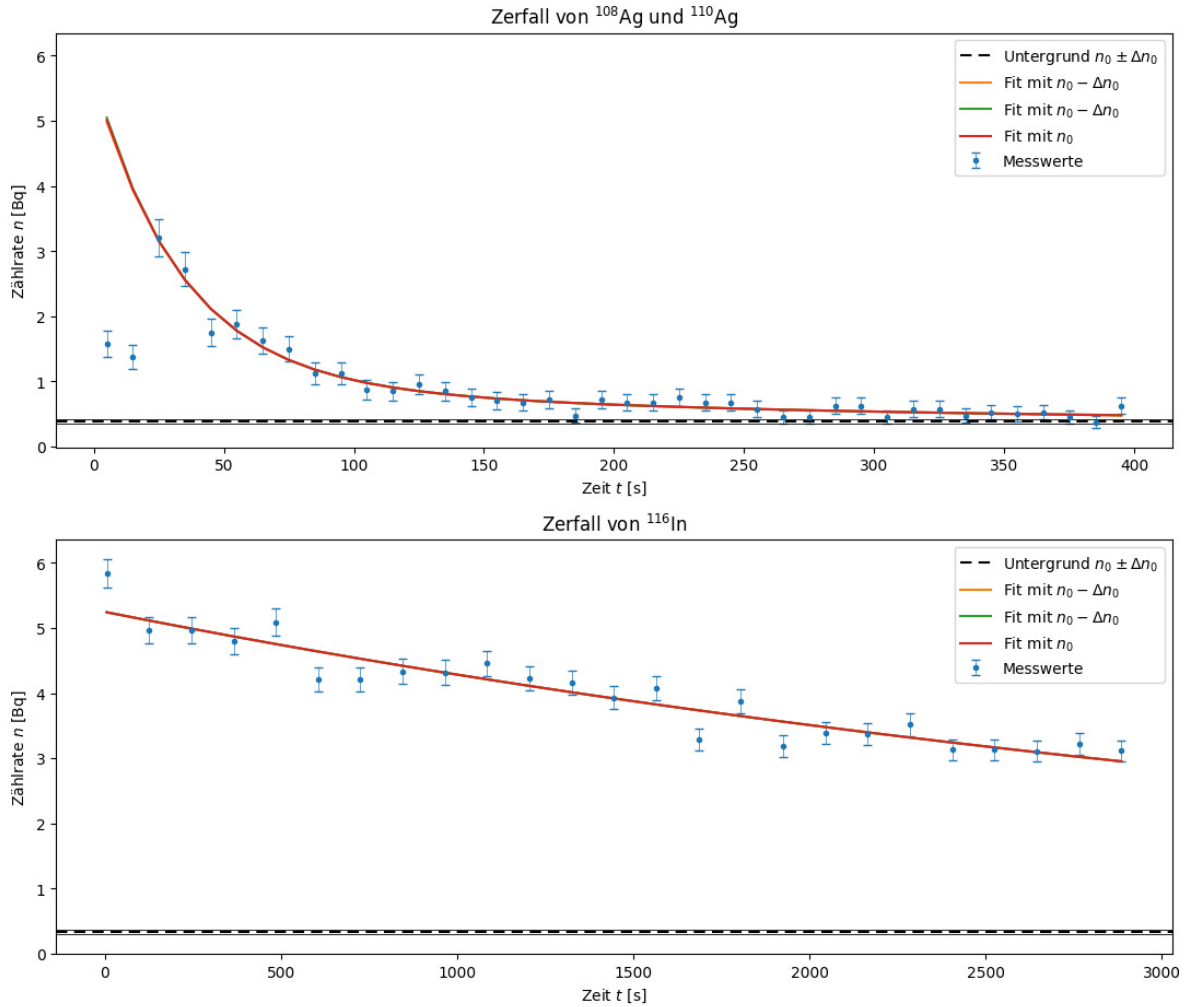


Abb. 4.2: Silber- und Indium Aktivität mit eingezeichnetem Untergrund

Man sieht durch die gleiche y-Achsen-Skalierung auch gut den direkten Vergleich, welche Auswirkung die Halbwertszeiten haben (schnelles Absinken bei Silber, ...).

4.5. Startaktivitäten

Eine gute Beobachtung kann noch aus diesen Diagrammen gewonnen werden: Die Aktivität beider radioaktiven Quellen sollte zu Beginn in derselben Größenordnung sein (beide Fits

beginnen bei demselben Wert *Junge ich kann nicht mehr, jetzt vergleiche ich halt doch die Startaktivitäten/Fitparameter*):

$$A_{110} = (4.6 \pm 0.9) \text{ Bq} \quad A_{108} = (0.7 \pm 0.4) \text{ Bq} \quad A_{116} = (4.6 \pm 0.9) \text{ Bq}$$

Die Startaktivität $A_{108} \ll A_{110}$ ist völlig valid, auf meinen ersten müden Blick war ich verwirrt, da die asymptotischen Sättigungskurven in Abb. 1.2 dem müden Beobachter richtigerweise zeigen, dass $A_{108}(t' \rightarrow \infty) = A_{110}(t' \rightarrow \infty) = A_\infty$, hier ist t' jedoch die Aktivierungszeit. Beim betrachteten Fit hingegen handelt es sich aber nicht mehr um zeitabhängige Aktivitäten während der Aktivierungsphase, sondern um die Konstanten A_{0110}, A_{0108} die nur noch den reinen Zerfall beschreiben und in meiner Auswertung aus Platz- und Schreibarbeitgründen stets nur als A_{110}, A_{108} abgekürzt wurden. Die Konsistenz des Fits kann jedoch überprüft werden mit:⁴

$$A(t) = \lambda N_0 e^{-\lambda t} = \overbrace{\frac{\ln 2}{T_{1/2}}}^{A_0} N_0 e^{-\lambda t} \quad (4.5)$$

~~N_0 sollte nach der Aktivierungsdauer von 7 min annähernd gleich sein, A_∞ ist auch gleich, was bei $t = 0$ bedeutet:~~

$$V_F := \frac{A_{0110}}{A_{0108}} = \frac{A_{110}}{A_{108}} \quad \begin{array}{c} \xrightarrow{t' \rightarrow \infty} \\ \text{---} \end{array} \quad \begin{array}{c} T_{108} \\ \text{---} \\ T_{110} \end{array} := V_T \quad \boxed{V_F = 6.6 \pm 4.0 \quad V_T = 5.9} \quad (4.6)$$

Hier wurden die Fitergebnisse mit dem Größenverhältnis der Literaturwerte für $T_{1/2}$ verglichen. ~~leichte Differenz macht auch Sinn, da ^{110}Ag bei der genutzten Aktivierungszeit schneller in Sättigung geht, also N_{0110} ein wenig größer als N_{0108} ist, dadurch steigt das Verhältnis der Startaktivitäten.~~ *Bemerkung:* Ich habe hier absichtlich nicht exakt auf die signifikante Stelle gerundet, da dann stark nach oben zu 7 ± 4 gerundet worden wäre.

4.6. **Update:** Sättigungsaktivität

Der vorherige Abschnitt hat sich nach Überprüfung durch eine kalifornische Serverfarm als falsch herausgestellt. Das Skript, speziell Abb. 1.2 suggeriert allen Beobachtern - nicht nur müden, dass $A_\infty^{108} = A_\infty^{110}$. Dies stimmt nicht, sondern es gilt, dass die Sättigungsaktivität abhängig des Neutronenflusses Φ , des Wirkungsquerschnitts σ des zu aktivierenden Isotops und der Anzahl an stabilen Isotopkernen N_{st} ist:⁵

$$A_\infty = N_{\text{st}} \sigma \Phi \implies \frac{A_{0110}}{A_{0108}} = \frac{A_\infty^{110}(1 - e^{-\lambda_{110} t'})}{A_\infty^{108}(1 - e^{-\lambda_{110} t'})} = \frac{\sigma_{109} N_{\text{st}}^{109}}{\sigma_{107} N_{\text{st}}^{107}} \frac{1 - \exp\left(-\frac{\ln 2}{T_{110}} t'\right)}{1 - \exp\left(-\frac{\ln 2}{T_{108}} t'\right)} \quad (4.7)$$

Nach der IAEA Database⁶ beträgt $\sigma_{107} = 34.8$ barn und $\sigma_{109} = 93.4$ barn sowie nach Wikipedia³ $N_{\text{st}}^{107} = 51.893\%$ und $N_{\text{st}}^{109} = 48.161\%$. Damit berechnet man: $\boxed{V_A = 2.97}$. Oben wurde eben

die falsche Annahme gemacht, dass $N_0^{110} = N_0^{108}$.

Interessant ist jetzt die hohe absolute Abweichung zum Verhältniswert der Fits von $V_F = 6.6 \pm 4.0$, der glücklicherweise einen hohen Fehler hat. Die kalifornische Serverfarm schlägt als Grund noch die unterschiedliche Ansprechwahrscheinlichkeit des GMZ für die Zerfallselektronen vor, da die Detektion von der Energie der Teilchen abhängig sei, beim Betazerfall von ^{110}Ag und ^{108}Ag werden unterschiedliche Energien frei. Dabei wird in Versuch 253: „Absorption von α, β, γ -Strahlung“ von einer Ansprechwahrscheinlichkeit von 1 für β -Strahlung gesprochen. Zusätzlich ist $A_{108} = 0.7 \pm 0.4$ sehr klein mit hohem Fehler, wodurch das Verhältnis $V_F \sim \frac{1}{A_{108}}$ bei kleinen Veränderungen stark schwankt.

Vergleich A_{110} mit A_{116} : Dass diese den denselben Wert haben, ist angesichts derselben Neutronenquelle, die zur Aktivierung verwendet wurde, völlig sinnvoll. (Unter der Voraussetzung, dass jedes Neutron der Neutronenquelle bei Silber als auch bei Indium mit derselben Wahrscheinlichkeit einen Atomkern aktiviert, der Wirkungsquerschnitt also derselbe ist. Das habe ich nicht nachgeschaut, irgendwann reicht's auch mal). Nur sind bei Silber schon während des Transports des aktivierten Präparats einige Kerne zerfallen, weswegen die real aufgenommenen Werte, die blauen Messpunkte, bei Silber niedriger als bei Indium sind.

4.7. aufmodulierte Störung

Bei beiden Messungen ist eine wiederholt auftretende Störung in den Diagrammen Abb. 3.1 und 3.2 sichtbar: Die Messwerte scheinen in Form von „Wellenstufen“ aufzutreten, vorstellbar wie eine aufmodulierte Sinuskurve, deren Trägerfrequenz die Exponentialfunktion ist. Diese Formen treten periodisch auf, deswegen ist ein statistischer Zufall meiner Einschätzung nach unwahrscheinlich, vielleicht interpretiere ich in die Messpunktverteilung aber auch zu viel hinein. Meine einzige Vermutung einer Ursache macht jedoch keinen Sinn, denn das Netzbrummen des Stromnetzes existiert auf einer viel kleineren Zeitskala als die über mehrere Minuten aufgenommenen Messwerte. Wäre aber interessant zu wissen, ob das auch in anderen Auswertungen zu beobachten ist...

4.8. Fazit

Zwar sind die Fits und damit die Fehlergrenzen der berechneten Halbwertszeiten relativ hoch, die absolute Abweichung zu den Literaturwerten ist jedoch stets gering. Das Experiment wird als Erfolg gewertet. War auch ein sehr entspannter Versuch, der durch schnelleres Arbeiten bei der Silbermessung wsl. noch zu deutlich besseren Ergebnissen führen wird. Zudem könnte man zur Verringerung des relativen Fehlers von Silber mehr als nur 4 Messreihen aufnehmen und für eine kleine Verbesserung länger aktivieren.

Die Auswertung war doch ganz amüsant (bis auf die Zeit, die es gekostet hat, die addierten Messreihen bei Silber wieder zu entfernen, sich über das Zustandekommen des relativen Fehlers

abhängig von Torzeit und Anzahl der Messreihen klarzuwerden, noch Gedanken über die Startaktivität zu machen), vor allem sind mir endlich mal wieder Memes eingefallen, auch wenn sie sie nicht immer nahtlos in den Kontext des Versuches eingefügt haben. Und ich verstehe jetzt, warum in diesem Versuch keine Frage zu A_0 gestellt wurde.



Abb. 4.3: Tage an denen ich PAP Versuche schreibe, starten morgens und enden erst in den frühen Morgenstunden und brauchen einen weiteren Zyklus¹⁰

Literaturquellen

- ¹Dr. J. Wagner, *Physikalisches Praktikum 2.2 für Studierende der Physik B.Sc.* [Online; Stand 04/2026] (Universität Heidelberg, Apr. 2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (besucht am 14.04.2026) (siehe S. 3, 5, 6, 8, 9, 12, 15).
- ²Wikipedia, *Kritikalität*, [Online; Stand 07/2026], <https://de.wikipedia.org/wiki/Kritikalit%C3%A4t> (siehe S. 6).
- ³Wikipedia, *Silber*, [Online; Stand 07/2026], <https://de.wikipedia.org/wiki/Silber> (siehe S. 7, 21).
- ⁴Wikipedia, *Aktivität (Physik)*, [Online; Stand 07/2026], [https://de.wikipedia.org/wiki/Aktivit%C3%A4t_\(Physik\)](https://de.wikipedia.org/wiki/Aktivit%C3%A4t_(Physik)) (siehe S. 21).
- ⁵Dr. Thomas Kormoll, *Physikalisches Grundpraktikum: Neutronenaktivierung*, [Online; Stand 07/2026] (Technische Universität Dresden, 07.05.2026), <https://tu-dresden.de/mn/physik/ressourcen/dateien/studium/lehrveranstaltungen/praktika/pdf/RM3.pdf?lang=de> (siehe S. 21).
- ⁶International Atomic Energy Agency, *Live Chart of Nuclides*, [Online; Stand 07/2026], <https://www-nds.iaea.org/relnsd/vcharthtml/VChartHTML.html> (siehe S. 21).

Abbildungsquellen

- ⁷Ludwig Göransson, Christopher Nolan, *Oppenheimer (soundtrack)*, [Movie, 2023], label: Back Lot; movie-production: Syncopy, Atlas Entertainment, <https://www.youtube.com/watch?v=4JZ-o3iAJv4> (siehe S. 1).
- ⁸Charles Levy, *atomic bomb on Nagasaki*, [Online; Stand 07/2026], <https://atomicphotographers.com/photographers/charles-levy/> (siehe S. 6).
- ⁹C. Nolan, *Interstellar*, [Movie, 2014], Paramount Pictures und Warner Bros. (siehe S. 17).
- ¹⁰Green Mile, *I'm tired boss*, [Online; Stand 07/2026], <https://knowyourmeme.com/memes/im-tired-boss> (siehe S. 23).

A. Code-Anhang

Bei den Funktionen habe ich teilweise Dinge von verschiedenen Altprotokollen zusammengesucht, und auf meine Bedürfnisse umgeschrieben. Die konkreten Berechnung sind dann eigenständig.

July 30, 2026

```
[1]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #L
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```

```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[2]: currentversuch = "252"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
    ↪ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\252

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\252\Diagramme

Delete UTF8 False encoding

```

[3]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a] '
#     text=re.sub(patterna, "ä", text)
#     patternu=r'[][u] '
#     text=re.sub(patternu, "ü", text)
#     patterno=r'[][o] '
#     text=re.sub(patterno, "ö", text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevowels(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'\-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)

    # Das Pattern sucht nach " gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'"([auoAUO])'", convert, text)

```

```

[4]: print(deletevowels("""
"""))

```

Runden signifikanter Stellen

```

[5]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran..
↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt..
↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter

```

```

-----
**val** : float
    Messwert

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""
einheiten_str = einheiten if einheiten is not None else ""
if errVal is not None:
    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal)))) # Die erste
    ↪signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so
    ↪verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
    ↪alle Nachkommastellen weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_CEILING) * scale

    num = f"\num{{{val_round}}({err_round})}"
    qty = f"\qty{{{val_round}}({err_round)}}{{{einheiten_str}}}" if
    ↪einheiten else num

    return float(val_round), float(err_round), num, qty
else:

```

```

    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↳erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    val_round = (val / scale).quantize(Decimal('1'),
↳rounding=ROUND_HALF_UP) * scale
    num = f"\\num{{{val_round}}}"
    qty = f"\\qty{{{val_round}}}{einheiten_str}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳excluded=['einheiten'])

    # wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳gewollt sind, die 10~/e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[6]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↳ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↳genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen

```

```

"""

error = 0
errProneParamaters = []

function = sp.sympify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable)           # Die Funktion
     $\hookrightarrow$  wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term))                       # Fehler werden
     $\hookrightarrow$  quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[7]: def sigma_abweichung(p1, err_p1,p2,err_p2):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
     $\hookrightarrow$  Messwert und einem Literaturwert.
    """
    if err_p1 == 0 and err_p2 == 0:
        raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
         $\hookrightarrow$  fehlerbehaftet sein!")
    else:
        abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

```

```

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
↪ Nachkommastellenposition hinzu
    exp -= 1
    scale = Decimal(10) ** exp
    abweichung_round = (abweichung / scale).quantize(Decimal('1'),
↪ rounding=ROUND_CEILING) * scale
    res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round), res

```

```

[8]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
↪ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
            Messreihe & {nam1}[\\unit{{{einheiten}}}] &
↪ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
↪ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
↪ err2_array):

        VAL1=round_to_sigs(val1,err1,r')[2]
        if err2!=0:

```

```

        VAL2=round_to_sigs(val2,err2,r'[2]
else:
    VAL2=val2

abs=np.abs(val1-val2)
abs_delta=np.sqrt(err1**2+err2**2)
if abs_delta!=0:
    SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
else:
    SIGMA=0.0

rel=abs/np.abs(val1)*100
if abs != 0:
    rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
    ABS = round_to_sigs(abs, abs_delta, r'[2]
    REL = round_to_sigs(rel, rel_delta, r'[2]
else:
    rel_delta = 0
    ABS = "0"
    REL = "0"

# Eine saubere Zeile nur mit den Werten für LaTeX
row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
table_rows.append(row)

body_str = "\\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{{nam1}}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[9]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]

```



```

        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r')[2]
        REL = round_to_sigs(rel, rel_delta, r')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZx}}
        \\toprule
        \\
        ↪{nam1}[\\unit{{einheiten}}]&{nam2}[\\unit{{einheiten}}]&\\text{{abs.Abw.
        ↪}}[\\unit{{einheiten}}]&\\text{{proz.Abw.
        ↪}}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)

```

```

[35]: N=[]
torzeit=10
for i in range(1,5):
    pfad=f"silber{i}.txt"
    n=np.loadtxt(Messwerte_path/"Silber"/pfad,
    ↪delimiter=",",usecols=[1],skiprows=4,unpack=True)
    N.append(n)
n=np.sum(N,axis=0)/(4*torzeit)
Delta_n=np.sqrt(np.sum(N,axis=0))/(4*torzeit)

relFehler0=np.mean(Delta_n/n)
relFehler1=1/2*np.mean(1/np.sqrt(n*torzeit))
relFehler2=1/2*1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)

```

```

print("relFehler:sqrt(T)",relFehler2)

t=np.arange(5,400,10)
background=np.loadtxt(Messwerte_path/"Silber"/"untergrund.txt",
    ↪delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/torzeit
Delta_background=np.std(background/torzeit)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1,A2,l2):
    return A1*np.exp(-x*l1) + A2*np.exp(-x*l2) + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0-\Delta n_0$",r"Fit mit
    ↪$n_0$",]
plt.close('all')
fig, ax1 = plt.subplots(1, 1, figsize=(figsize_cm(14,6)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
    ↪5,label='Messwerte')# ecolor='black'
ax1.set_title(r'Zerfall von  $^{108}\mathrm{Ag}$  und  $^{110}\mathrm{Ag}$ ')
ax1.set_xlabel(r'Zeit $t$ [s]')
ax1.set_ylabel(r'Zählrate $n$;[\text{Bq}]$')
ax1.set_yscale('log')
# ax1.axhline(mean_background,linestyle=(0, (5,
    ↪3)),color='black',label=r'Untergrund $n_0\pm\Delta n_0$')
# ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
# ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
lambda2=[]
Delta_lambda1=[]
Delta_lambda2=[]
A1=[]
A2=[]
Delta_A1=[]
Delta_A2=[]

for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t[2:],n[2:], p0=[500,0.02,50,0.
    ↪002],sigma=Delta_n[2:], absolute_sigma=True)
    if i==2:
        poptspeicher=popt
    ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
    lambda1.append(popt[1])
    Delta_lambda1.append(np.sqrt(pcov[1][1]))
    lambda2.append(popt[3])

```

```

Delta_lambda2.append(np.sqrt(pcov[3][3]))

A1.append(popt[0])
Delta_A1.append(np.sqrt(pcov[0][0]))
A2.append(popt[2])
Delta_A2.append(np.sqrt(pcov[2][2]))
print(poptspeicher)
print("lambda1=",lambda1)
print("Delta_lambda1=",Delta_lambda1)
print("lambda2=",lambda2)
print("Delta_lambda2=",Delta_lambda2)

Delta_l1=np.sqrt(np.mean([np.abs(lambda1[2]-lambda1[0]),np.
↪abs(lambda1[2]-lambda1[1])])**2+Delta_lambda1[2]**2)
l1,Delta_l1,_,latexl1=round_to_sigs(lambda1[2],Delta_l1,r'\per\s')
print("l1=",latexl1)
Delta_l2=np.sqrt(np.mean([np.abs(lambda2[2]-lambda2[0]),np.
↪abs(lambda2[2]-lambda2[1])])**2+Delta_lambda2[2]**2)
l2,Delta_l2,_,latexl2=round_to_sigs(lambda2[2],Delta_l2,r'\per\s')
print("l2=",latexl2)

DELTA_A1=np.sqrt(np.mean([np.abs(A1[2]-A1[0]),np.
↪abs(A1[2]-A1[1])])**2+Delta_A1[2]**2)
_,_,_,latexA1=round_to_sigs(A1[2],DELTA_A1,r'\per\s')
print("A1=",latexA1)
DELTA_A2=np.sqrt(np.mean([np.abs(A2[2]-A2[0]),np.
↪abs(A2[2]-A2[1])])**2+Delta_A2[2]**2)
_,_,_,latexA2=round_to_sigs(A2[2],DELTA_A2,r'\per\s')
print("A2=",latexA2)

T_1=np.log(2)/l1
T_2=np.log(2)/l2
Delta_T_1=np.log(2)*Delta_l1/l1**2
Delta_T_2=np.log(2)*Delta_l2/l2**2
T_1,Delta_T_1,_,latexT1=round_to_sigs(T_1,Delta_T_1,r'\s')
T_2,Delta_T_2,_,latexT2=round_to_sigs(T_2,Delta_T_2,r'\s')
print("T1=",latexT1)
print("T2=",latexT2)

tau1=1/l1
Delta_tau1=Delta_l1/l1**2
tau1,Delta_tau1,_,latextau1=round_to_sigs(tau1,Delta_tau1,r'\s')
print("tau1=",latextau1)
tau2=2/l2
Delta_tau2=Delta_l2/l2**2
tau2,Delta_tau2,_,latextau2=round_to_sigs(tau2,Delta_tau2,r'\s')
print("tau2=",latextau2)

```

```

chi2_f=np.sum((fit_func(t[2:],*poptspeicher)-n[2:])**2/Delta_n[2:]**2)
dof_f=len(n[2:])-4 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f
print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=",round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

ax1.legend()
# ax1.grid()

plt.tight_layout()
# plt.savefig(Ausgabe_path/'silberlog.png',format="png",bbox_inches='tight')
plt.show()

# m,Delta_m,_,latexm=round_to_sigs(popt[0],np.
#   ↳sqrt(pcov[0][0]),r'\volt\per\hertz')
# print("links:")
# print(f"m={latexm}")
# print("c="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\milli\volt')[3])
# print("rechts:")
# print(r"m="+round_to_sigs(poptI[1],np.
#   ↳sqrt(pcovI[0][0]),r'\milli\volt\per\ampere')[3])
# print(r"c="+round_to_sigs(poptI[0],np.sqrt(pcovI[1][1]),r'\milli\volt')[3])

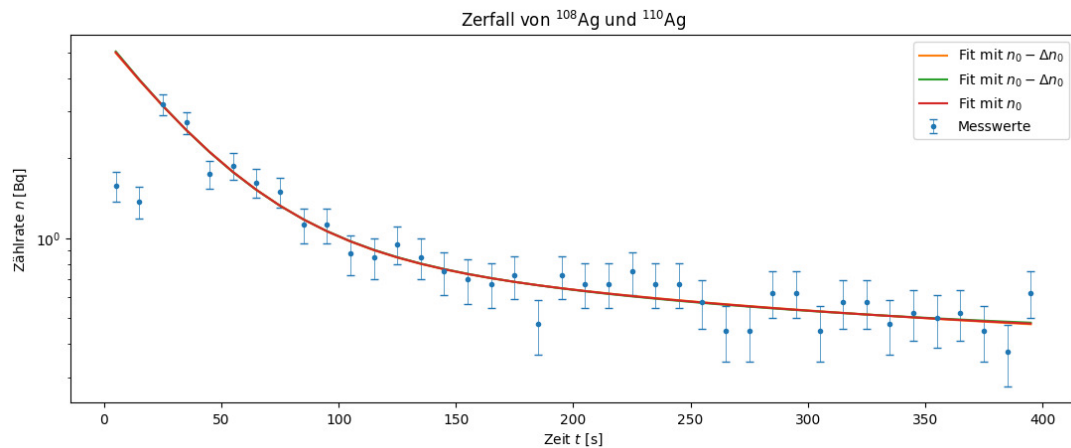
```

```

relFehler:deltan/n) 0.18346787481425433
relFehler:sqrt(N) 0.18346787481425433
relFehler:sqrt(T) 0.18346787481425436
background 0.38125
[4.63648739 0.02990991 0.65241358 0.00483349]
lambda1= [np.float64(0.029379146937190723), np.float64(0.03073100866797658),
np.float64(0.029909914269914094)]
Delta_lambda1= [np.float64(0.007035389142916432),
np.float64(0.008852347110782243), np.float64(0.007713908799220168)]
lambda2= [np.float64(0.00408575179595493), np.float64(0.005891853523965589),
np.float64(0.004833485902578989)]
Delta_lambda2= [np.float64(0.0018668422450091556),
np.float64(0.0026884436966492344), np.float64(0.0022041577033376837)]
l1= \qty{0.030(0.008)}{\per\s}
l2= \qty{0.0048(0.0024)}{\per\s}
A1= \qty{4.6(0.9)}{\per\s}
A2= \qty{0.7(0.4)}{\per\s}
T1= \qty{23(7)}{\s}
T2= \qty{140(80)}{\s}
tau1= \qty{33(9)}{\s}
tau2= \qty{420(110)}{\s}

```

```
chi2= 19.0
chi2_red= 0.6
Wahrscheinlichkeit= 98.0 %
```



```
[34]: torzeit=120
N=np.loadtxt(Messwerte_path/"indium.txt",
    ↪delimiter=",",usecols=[1],skiprows=4,unpack=True)
n=N/torzeit
Delta_n=np.sqrt(N)/torzeit
t=np.arange(5,3000,120)
relFehler0=np.mean(Delta_n/n)
relFehler1=np.mean(1/np.sqrt(N))
relFehler2=1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)
print("relFehler:sqrt(T)",relFehler2)

background=np.loadtxt(Messwerte_path/"untergrund_indium.txt",
    ↪delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/10
Delta_background=np.std(background/10)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1):
    return A1*np.exp(-x*l1) + + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit $n_0-\Delta n_0$",r"Fit mit $n_0-\Delta n_0$",r"Fit mit_
    ↪$n_0$",]

plt.close('all')
```

```

fig, ax1 = plt.subplots(1, 1, figsize=(figsize_cm(14,6)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
↳5,label='Messwerte')# ecolord='black'
ax1.set_title(r'Zerfall von  $^{116}\mathrm{In}$ ')
ax1.set_xlabel(r'Zeit  $t$  [s]')
ax1.set_ylabel(r'Zählrate  $n$  [Bq]')
ax1.set_yscale('log')
# ax1.axhline(mean_background,linestyle=(0, (5,↳
↳3)),color='black',label=r'Untergrund  $n_0 \pm \Delta n_0$ ')
# ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
# ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
Delta_lambda1=[]
for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t,n, p0=[500,0.02],sigma=Delta_n,↳
↳absolute_sigma=True)
    if i==2:
        poptspeicher=popt
        ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
        lambda1.append(popt[1])
        Delta_lambda1.append(np.sqrt(pcov[1][1]))
Delta_l1=np.sqrt(np.mean([np.abs(lambda1[2]-lambda1[0]),np.
↳abs(lambda1[2]-lambda1[1]))**2+Delta_lambda1[2]**2)
l1,Delta_l1,_,latexl1=round_to_sigs(lambda1[2],Delta_l1,r'\per{s}')
print("l1=",latexl1)
DELTA_A1=np.sqrt(np.mean([np.abs(A1[2]-A1[0]),np.
↳abs(A1[2]-A1[1]))**2+Delta_A1[2]**2)
_,_,_,latexA1=round_to_sigs(A1[2],DELTA_A1,r'\per{s}')
print("A1=",latexA1)

T_1=np.log(2)/l1
Delta_T_1=np.log(2)*Delta_l1/l1**2
T_1,Delta_T_1,_,latexT1=round_to_sigs(T_1,Delta_T_1,r'\s')
print("T1",latexT1)

tau=1/l1
Delta_tau=Delta_l1/l1**2
tau,Delta_tau,_,latextau=round_to_sigs(tau,Delta_tau,r'\s')
print("tau",latextau)

chi2_f=np.sum((fit_func(t[2:],*poptspeicher)-n[2:]**2/Delta_n[2:]**2)
dof_f=len(n[2:])-2 #dof:degrees of freedom, Freiheitsgrad
chi2_red=chi2_f/dof_f

```

```

print("chi2=", round_to_sigs(chi2_f)[0])
print("chi2_red=", round_to_sigs(chi2_red)[0])
prob=round(1-chi2.cdf(chi2_f,dof_f),2)*100
print("Wahrscheinlichkeit=", prob,"%")

ax1.legend()
# ax1.grid()

plt.tight_layout()
# plt.savefig(Ausgabe_path/'indiumlog.png',format="png",bbox_inches='tight')
plt.show()

# m,Delta_m,_,latexm=round_to_sigs(popt[0],np.
#   ↳sqrt(pcov[0][0]),r'\volt\per\hertz')
# print("links:")
# print(f"m={latexm}")
# print("c="+round_to_sigs(popt[1],np.sqrt(pcov[1][1]),r'\milli\volt')[3])
# print("rechts:")
# print(r"m="+round_to_sigs(poptI[1],np.
#   ↳sqrt(pcovI[0][0]),r'\milli\volt\per\ampere')[3])
# print(r"c="+round_to_sigs(poptI[0],np.sqrt(pcovI[1][1]),r'\milli\volt')[3])

```

relFehler:deltan/n) 0.04622544934289126

relFehler:sqrt(N) 0.04622544934289126

relFehler:sqrt(T) 0.046225449342891245

background 0.3333333333333337

$l_1 = \sqrt[0.000218(0.000012)]{\text{per}\text{s}}$

$A_1 = \sqrt[4.6(0.9)]{\text{per}\text{s}}$

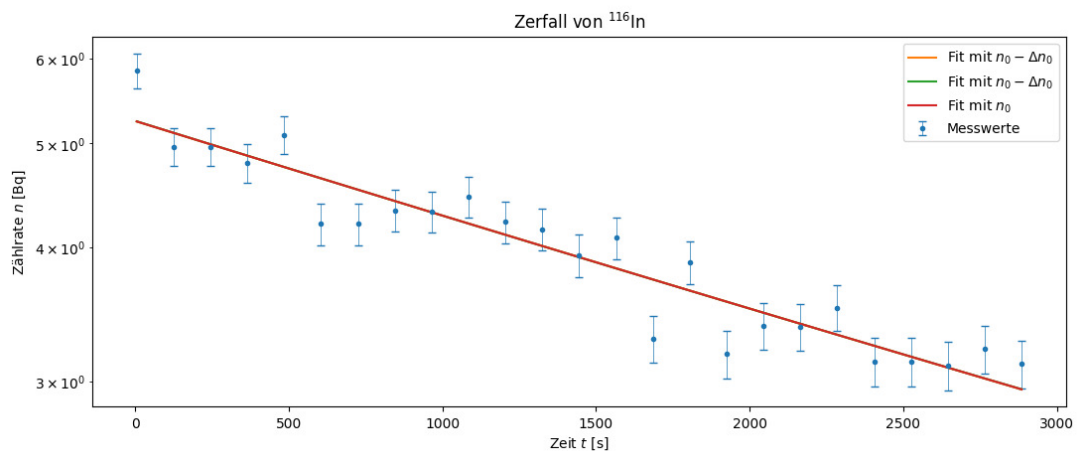
$T_1 \sqrt[3180(180)]{\text{s}}$

$\tau \sqrt[4600(300)]{\text{s}}$

chi2= 30.0

chi2_red= 1.7

Wahrscheinlichkeit= 3.0 %




```

[33]: N=[]
torzeit=10
for i in range(1,5):
    pfad=f"silber{i}.txt"
    n=np.loadtxt(Messwerte_path/"Silber"/pfad,␣
    ↪delimiter=",",usecols=[1],skiprows=4,unpack=True)
    N.append(n)
n=np.sum(N,axis=0)/(4*torzeit)
Delta_n=np.sqrt(np.sum(N,axis=0))/(4*torzeit)

t=np.arange(5,400,10)
background=np.loadtxt(Messwerte_path/"Silber"/"untergrund.txt",␣
    ↪delimiter=",",skiprows=4,usecols=[1])
mean_background=np.mean(background)/torzeit
Delta_background=np.std(background/torzeit)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1,A2,l2):
    return A1*np.exp(-x*l1) + A2*np.exp(-x*l2) + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])
plotlabel=[r"Fit mit  $n_0-\Delta n_0$ ",r"Fit mit  $n_0-\Delta n_0$ ",r"Fit mit␣
    ↪ $n_0$ ",]

plt.close('all')
fig, (ax1,ax2) = plt.subplots(2, 1, sharey=True,figsize=(figsize_cm(14,12)))
# fig.suptitle(r'Silbermessung')
ax1.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
    ↪5,label='Messwerte')# ecolor='black'
ax1.set_title(r'Zerfall von  $^{108}\mathrm{Ag}$  und  $^{110}\mathrm{Ag}$ ')
ax1.set_xlabel(r'Zeit  $t$  [s]')
ax1.set_ylabel(r'Zählrate  $n$ ; [ $\text{Bq}$ ])')
ax1.set_yscale('log')
ax1.axhline(mean_background,linestyle=(0, (5,␣
    ↪3)),color='black',label=r'Untergrund  $n_0\pm\Delta n_0$ ')
ax1.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
ax1.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

lambda1=[]
lambda2=[]
Delta_lambda1=[]
Delta_lambda2=[]
for i in range(3):
    y_0=y_0array[i]

```

```

    popt, pcov=curve_fit(fit_func,t[2:],n[2:], p0=[500,0.02,50,0.
↳002],sigma=Delta_n[2:], absolute_sigma=True)
    if i==2:
        poptspeicher=popt
        ax1.plot(t,fit_func(t,*popt),label=plotlabel[i])
        lambda1.append(popt[1])
        Delta_lambda1.append(np.sqrt(pcov[1][1]))
        lambda2.append(popt[3])
        Delta_lambda2.append(np.sqrt(pcov[3][3]))

ax1.legend()
# ax1.grid()

torzeit=120
N=np.loadtxt(Messwerte_path/"indium.txt",
↳delimiter=" ",usecols=[1],skiprows=4,unpack=True)
n=N/torzeit
Delta_n=np.sqrt(N)/torzeit
t=np.arange(5,3000,120)
relFehler0=np.mean(Delta_n/n)
relFehler1=np.mean(1/np.sqrt(N))
relFehler2=1/sqrt(torzeit)*np.mean(1/np.sqrt(n))
print("relFehler:deltan/n",relFehler0)
print("relFehler:sqrt(N)",relFehler1)
print("relFehler:sqrt(T)",relFehler2)

background=np.loadtxt(Messwerte_path/"untergrund_indium.txt",
↳delimiter=" ",skiprows=4,usecols=[1])
mean_background=np.mean(background)/10
Delta_background=np.std(background/10)/np.sqrt(len(background)-1)
print("background",mean_background)

def fit_func(x, A1,l1):
    return A1*np.exp(-x*l1) + + y_0
y_0array=mean_background+np.array([-Delta_background,Delta_background,0])

ax2.errorbar(t,n,yerr=Delta_n,fmt='.',capsize= 3,elinewidth= 0.
↳5,label='Messwerte')# ecolor='black'
ax2.set_title(r'Zerfall von  $^{116}\mathrm{In}$ ')
ax2.set_xlabel(r'Zeit $t$ [s]')
ax2.set_ylabel(r'Zählrate $n$ [Bq]')
ax2.set_yscale('log')
ax2.axhline(mean_background,linestyle=(0, (5,
↳3)),color='black',label=r'Untergrund $n_0\pm\Delta n_0$')
ax2.axhline(mean_background-Delta_background,linewidth=0.6,color='black',)
ax2.axhline(mean_background+Delta_background,linewidth=0.6,color='black',)

```

```

lambda1=[]
Delta_lambda1=[]
for i in range(3):
    y_0=y_0array[i]
    popt, pcov=curve_fit(fit_func,t,n, p0=[500,0.02],sigma=Delta_n,
↪absolute_sigma=True)
    if i==2:
        poptspeicher=popt
        ax2.plot(t,fit_func(t,*popt),label=plotlabel[i])
        lambda1.append(popt[1])
        Delta_lambda1.append(np.sqrt(pcov[1][1]))

ax2.legend()

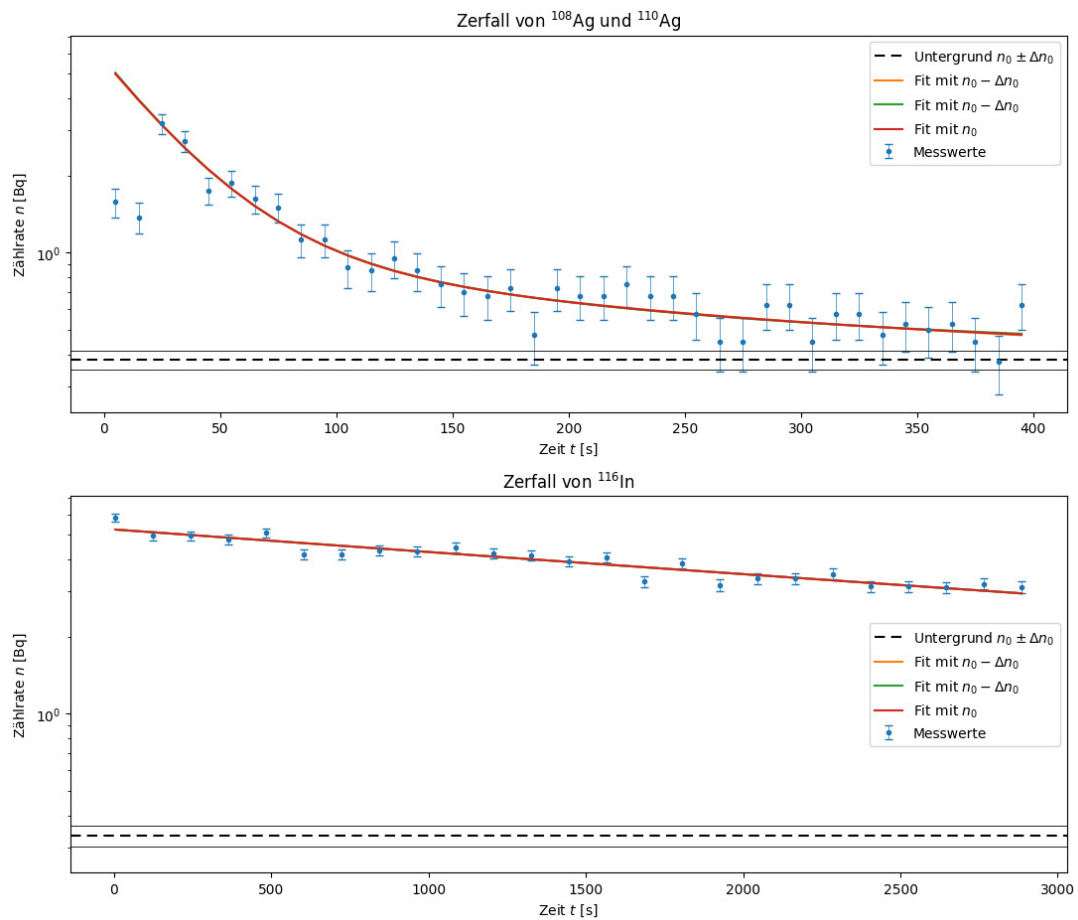
plt.tight_layout()
plt.savefig(Ausgabe_path/'silber_indiumlog.
↪png',format="png",bbox_inches='tight')
plt.show()

```

```

background 0.38125
relFehler:deltan/n) 0.04622544934289126
relFehler:sqrt(N) 0.04622544934289126
relFehler:sqrt(T) 0.046225449342891245
background 0.3333333333333337

```



```
[13]: compare_table_array(r'T_{\nicefrac{1}{2},\text{exp}}',r'T_{\nicefrac{1}{2},\text{Lit}}',r's',
    ↪41*60,24.6],[0,0])
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $T_{\{\frac{1}{2},\text{exp}\}}$  und
 $T_{\{\frac{1}{2},\text{Lit}\}}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe &  $T_{\{\frac{1}{2},\text{exp}\}}$ [\unit{s}] &
 $T_{\{\frac{1}{2},\text{Lit}\}}$ [\unit{s}] & \text{abs.Abw.}[\unit{s}] &
\text{proz.Abw.}[\unit{percent}] &  $S[\sigma]$  \\ \midrule
& \num{140(80)} & 144.60000000000002 & \num{0(80)} & \num{0(60)} &
\num{0.06} \\
& \num{23(7)} & 24.6 & \num{2(7)} & \num{10(40)} & \num{0.23} \\
\bottomrule
\end{tabularx}
\label{table:Vergleich_T_{\nicefrac{1}{2},\text{exp}}}
```

```
\end{table}
```

```
[14]: compare_table(r'T_{\nicefrac{1}{2}},\text{exp}}',r'T_{\nicefrac{1}{2}},\text{Lit}}',r'\s',3180,1
      ↪14*60,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von $T_{\{\nicefrac{1}{2}\}},\text{exp}}$ und
$T_{\{\nicefrac{1}{2}\}},\text{Lit}}$}
\begin{tabularx}{\textwidth}{ZZZx}
\toprule
T_{\{\nicefrac{1}{2}\}},\text{exp}}[\unit{\s}]&T_{\{\nicefrac{1}{2}\}},\text{Lit}}
[\unit{\s}]&\text{abs. Abw.}[\unit{\s}]&\text{proz. Abw.}[\unit{\percent}]&S[\unit{\sigm
a}]\midrule
\num{3180(180)}&\num{3248.4}&\num{70(180)}&\num{2(6)}&\num{0.4}
\bottomrule
\end{tabularx}
\label{table:Vergleich_T_{\{\nicefrac{1}{2}\}},\text{exp}}}
\end{table}
```

```
[15]: gff("a/b", "a,b")
```

Funktion:

$$\frac{a}{b}$$

```
\frac{a}{b}
```

Absoluter Fehler:

$$\sqrt{\frac{1}{b^4} (\Delta_a^2 b^2 + \Delta_b^2 a^2)}$$

```
\sqrt{\frac{\Delta_{a}^2 b^2 + \Delta_{b}^2 a^2}{b^4}}
```

Relativer Fehler:

$$\sqrt{\frac{\Delta_a^2}{a^2} + \frac{\Delta_b^2}{b^2}}$$

```
\sqrt{\frac{\Delta_{a}^2}{a^2} + \frac{\Delta_{b}^2}{b^2}}
```

```
[15]: (a/b,
      sqrt((Delta_a**2*b**2 + Delta_b**2*a**2)/b**4),
      sqrt(Delta_a**2/a**2 + Delta_b**2/b**2),
      [(a, Delta_a), (b, Delta_b)])
```

```
[ ]: A_0_110=4.6 # 110
      A_0_108=0.7 # 108
      Delta_A_0_110=0.9
      Delta_A_0_108=0.4
```

```

T_12=np.array([24.6,144.6]) # 110, 108

VA=A_0_110/A_0_108 # 110/108
Delta_VA=VA*np.sqrt((Delta_A_0_110/A_0_110)**2+(Delta_A_0_108/A_0_108)**2)
print(VA,Delta_VA)
VT=T_12[1]/T_12[0] #108/110 da A_0\sim 1/T
print(VA,VT)
t=420
V_A=(93.4*49*(1-np.exp(-np.log(2)/T_12[0]*t)))/(34.8*51*(1-np.exp(-np.log(2)/
↵T_12[1]*t))) # 110/108
print(V_A)

```

```

6.571428571428571 3.969112314036067
6.571428571428571 5.878048780487804
5.749999999999999
2.9760904695730965

```